

```
!pip install -q kaggle
```

```
import os
```

```
# Place kaggle.json in the expected location
os.makedirs("/root/.kaggle", exist_ok=True)
os.system("cp kaggle.json /root/.kaggle/")
os.system("chmod 600 /root/.kaggle/kaggle.json")
```

```
256
```

```
# Download Intel Image Classification dataset
os.system("kaggle datasets download -d puneet6060/intel-image-classification")
os.system("unzip -q intel-image-classification.zip -d intel_data")
```

```
0
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
import matplotlib.pyplot as plt
import time

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")
```

```
Using device: cuda
```

```
TRAIN_DIR = "intel_data/seg_train/seg_train"
VAL_DIR   = "intel_data/seg_test/seg_test"

NUM_CLASSES = 6      # buildings, forest, glacier, mountain, sea, street
BATCH_SIZE  = 64
EPOCHS      = 10
LR          = 1e-3
```

```
# AlexNet/VGG expect 224x224 inputs
train_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                        [0.229, 0.224, 0.225]),
])

val_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                        [0.229, 0.224, 0.225]),
])

train_ds = datasets.ImageFolder(TRAIN_DIR, transform=train_tf)
val_ds   = datasets.ImageFolder(VAL_DIR,   transform=val_tf)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
val_loader   = DataLoader(val_ds,   batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

print(f"Classes : {train_ds.classes}")
print(f"Train   : {len(train_ds)} images")
print(f"Val     : {len(val_ds)} images")

Classes : ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
Train    : 14034 images
Val      : 3000 images
```

```
def build_alexnet(num_classes, pretrained=True):
    """AlexNet with custom classifier head."""
    weights = models.AlexNet_Weights.IMAGENET1K_V1 if pretrained else None
    model = models.alexnet(weights=weights)
    # Replace final FC layer
    model.classifier[6] = nn.Linear(4096, num_classes)
    return model

def build_vgg16(num_classes, pretrained=True):
    """VGG-16 with batch norm, custom classifier head."""
    weights = models.VGG16_BN_Weights.IMAGENET1K_V1 if pretrained else None
    model = models.vgg16_bn(weights=weights)
    model.classifier[6] = nn.Linear(4096, num_classes)
    return model

def build_vgg19(num_classes, pretrained=True):
    """VGG-19 with batch norm, custom classifier head."""
    weights = models.VGG19_BN_Weights.IMAGENET1K_V1 if pretrained else None
    model = models.vgg19_bn(weights=weights)
    model.classifier[6] = nn.Linear(4096, num_classes)
    return model
```

```
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * imgs.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += imgs.size(0)
    return total_loss / total, correct / total

def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for imgs, labels in loader:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            outputs = model(imgs)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * imgs.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += imgs.size(0)
    return total_loss / total, correct / total

def train_model(name, model, epochs=EPOCHS, lr=LR):
    """Full training loop with history tracking."""
    print(f"\n{' '*55}")
    print(f"  Training {name}")
    print(f"{' '*55}")

    model = model.to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=4, gamma=0.5)

    history = {"train_loss": [], "val_loss": [],
              "train_acc": [], "val_acc": []}

    for epoch in range(1, epochs + 1):
        t0 = time.time()
        tr_loss, tr_acc = train_one_epoch(model, train_loader, criterion, optimizer)
        va_loss, va_acc = evaluate(model, val_loader, criterion)
        scheduler.step()
        elapsed = time.time() - t0
```

```

history["train_loss"].append(tr_loss)
history["val_loss"].append(va_loss)
history["train_acc"].append(tr_acc)
history["val_acc"].append(va_acc)

print(f"Epoch {epoch:02d}/{epochs}  "
      f"Train loss: {tr_loss:.4f}  acc: {tr_acc*100:.2f}% |  "
      f"Val loss: {va_loss:.4f}  acc: {va_acc*100:.2f}%  "
      f"({elapsed:.1f}s)")

return model, history

```

```
results = {}
```

```
Alexnet, results["AlexNet"] = train_model(
    "AlexNet", build_alexnet(NUM_CLASSES))
```

```
vgg16, results["VGG-16"] = train_model(
    "VGG-16", build_vgg16(NUM_CLASSES))
```

```
vgg19, results["VGG-19"] = train_model(
    "VGG-19", build_vgg19(NUM_CLASSES))
```

Downloading: "<https://download.pytorch.org/models/alexnet-owt-7be5be79.pth>" to /root/.cache/torch/hub/checkpoints/alexnet-owt-7be5be79.pth [100%] | 233M/233M [00:01<00:00, 186MB/s]

```
=====
Training AlexNet
```

```

Epoch 01/10 Train loss: 1.3721 acc: 42.38% | Val loss: 1.1998 acc: 57.23% (62.2s)
Epoch 02/10 Train loss: 1.0405 acc: 61.46% | Val loss: 0.9648 acc: 63.30% (60.0s)
Epoch 03/10 Train loss: 0.7792 acc: 72.16% | Val loss: 0.6931 acc: 74.83% (58.9s)
Epoch 04/10 Train loss: 0.6757 acc: 76.20% | Val loss: 0.4854 acc: 83.47% (60.5s)
Epoch 05/10 Train loss: 0.5158 acc: 81.84% | Val loss: 0.4304 acc: 84.73% (61.2s)
Epoch 06/10 Train loss: 0.4566 acc: 84.24% | Val loss: 0.4061 acc: 86.70% (59.3s)
Epoch 07/10 Train loss: 0.4092 acc: 85.85% | Val loss: 0.3818 acc: 87.73% (60.9s)
Epoch 08/10 Train loss: 0.4002 acc: 86.24% | Val loss: 0.3606 acc: 87.77% (60.0s)
Epoch 09/10 Train loss: 0.3188 acc: 88.96% | Val loss: 0.3374 acc: 88.80% (59.7s)
Epoch 10/10 Train loss: 0.2921 acc: 89.66% | Val loss: 0.3111 acc: 89.67% (61.0s)

```

Downloading: "https://download.pytorch.org/models/vgg16_bn-6c64b313.pth" to /root/.cache/torch/hub/checkpoints/vgg16_bn-6c64b313.pth [100%] | 528M/528M [00:06<00:00, 86.0MB/s]

```
=====
Training VGG-16
```

```

Epoch 01/10 Train loss: 0.6731 acc: 76.81% | Val loss: 0.7692 acc: 72.63% (250.9s)
Epoch 02/10 Train loss: 0.4341 acc: 85.31% | Val loss: 0.4729 acc: 83.17% (250.1s)
Epoch 03/10 Train loss: 0.3764 acc: 87.47% | Val loss: 0.3627 acc: 87.53% (250.4s)
Epoch 04/10 Train loss: 0.3647 acc: 87.54% | Val loss: 0.4446 acc: 84.90% (250.3s)
Epoch 05/10 Train loss: 0.2806 acc: 90.55% | Val loss: 0.2821 acc: 90.50% (250.6s)
Epoch 06/10 Train loss: 0.2388 acc: 91.69% | Val loss: 0.2295 acc: 92.10% (250.3s)
Epoch 07/10 Train loss: 0.2190 acc: 92.35% | Val loss: 0.2759 acc: 90.50% (250.7s)
Epoch 08/10 Train loss: 0.2272 acc: 92.18% | Val loss: 0.2631 acc: 90.93% (250.1s)
Epoch 09/10 Train loss: 0.1799 acc: 93.66% | Val loss: 0.2446 acc: 91.83% (250.5s)
Epoch 10/10 Train loss: 0.1661 acc: 93.95% | Val loss: 0.2371 acc: 91.63% (250.4s)

```

Downloading: "https://download.pytorch.org/models/vgg19_bn-c79401a0.pth" to /root/.cache/torch/hub/checkpoints/vgg19_bn-c79401a0.pth [100%] | 548M/548M [00:06<00:00, 88.4MB/s]

```
=====
Training VGG-19
```

```

Epoch 01/10 Train loss: 0.6812 acc: 77.49% | Val loss: 0.7295 acc: 77.43% (291.7s)
Epoch 02/10 Train loss: 0.4636 acc: 84.42% | Val loss: 0.6080 acc: 81.23% (292.0s)
Epoch 03/10 Train loss: 0.4013 acc: 86.65% | Val loss: 0.4423 acc: 85.13% (301.0s)
Epoch 04/10 Train loss: 0.3844 acc: 87.22% | Val loss: 0.4607 acc: 83.93% (300.8s)
Epoch 05/10 Train loss: 0.2908 acc: 90.05% | Val loss: 0.3405 acc: 88.33% (301.0s)
Epoch 06/10 Train loss: 0.2565 acc: 91.44% | Val loss: 0.2688 acc: 90.53% (300.2s)
Epoch 07/10 Train loss: 0.2496 acc: 91.61% | Val loss: 0.3259 acc: 88.93% (300.9s)
Epoch 08/10 Train loss: 0.2395 acc: 91.65% | Val loss: 0.3684 acc: 87.67% (300.9s)
Epoch 09/10 Train loss: 0.2004 acc: 93.05% | Val loss: 0.2408 acc: 91.40% (300.4s)
Epoch 10/10 Train loss: 0.1850 acc: 93.61% | Val loss: 0.2427 acc: 91.83% (300.7s)

```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
colors = {"AlexNet": "#e74c3c", "VGG-16": "#2ecc71", "VGG-19": "#3498db"}
```

```

for name, hist in results.items():
    epochs_range = range(1, len(hist["train_acc"]) + 1)
    axes[0].plot(epochs_range, [a*100 for a in hist["val_acc"]],

```

```

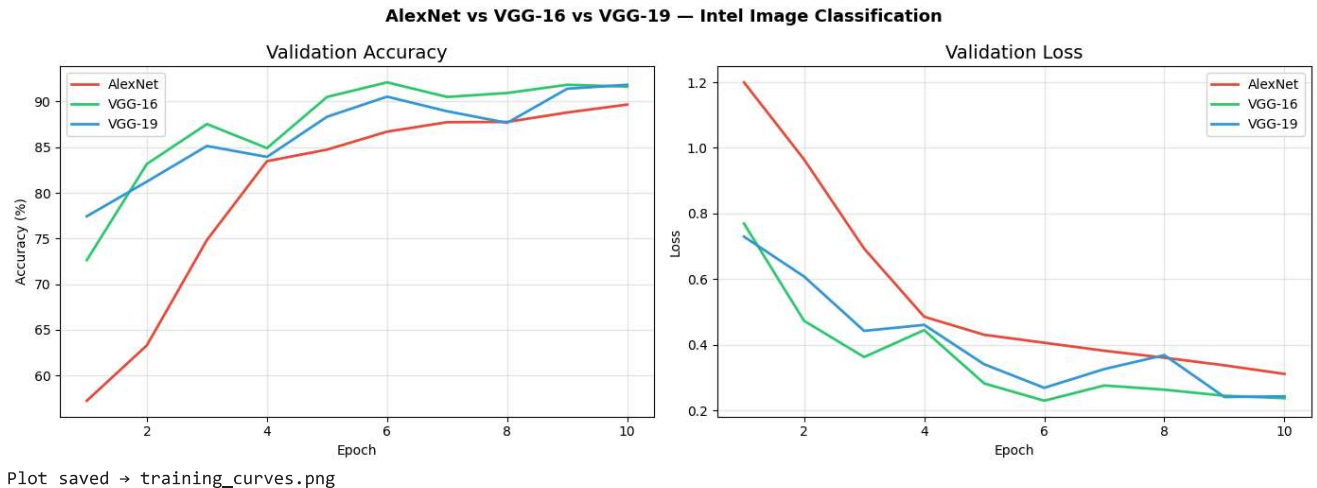
        label=name, color=colors[name], linewidth=2)
    axes[1].plot(epochs_range, hist["val_loss"],
        label=name, color=colors[name], linewidth=2)

    axes[0].set_title("Validation Accuracy", fontsize=14)
    axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Accuracy (%)")
    axes[0].legend(); axes[0].grid(alpha=0.3)

    axes[1].set_title("Validation Loss", fontsize=14)
    axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Loss")
    axes[1].legend(); axes[1].grid(alpha=0.3)

    plt.suptitle("AlexNet vs VGG-16 vs VGG-19 — Intel Image Classification",
        fontsize=13, fontweight="bold")
    plt.tight_layout()
    plt.savefig("training_curves.png", dpi=150)
    plt.show()
    print("Plot saved → training_curves.png")

```



```

print("\n" + "="*45)
print(f"{'Model':<12} {'Val Acc':>10} {'Val Loss':>10}")
print("-"*45)
for name, hist in results.items():
    best_acc = max(hist["val_acc"]) * 100
    best_loss = min(hist["val_loss"])
    print(f"{name:<12} {best_acc:>9.2f}% {best_loss:>10.4f}")
print("="*45)

```

```

=====
Model          Val Acc  Val Loss
-----
AlexNet        89.67%   0.3111
VGG-16         92.10%   0.2295
VGG-19         91.83%   0.2408
=====

```

```

torch.save(Alexnet.state_dict(), "alexnet_intel.pth")
torch.save(vgg16.state_dict(), "vgg16_intel.pth")
torch.save(vgg19.state_dict(), "vgg19_intel.pth")
print("\nModels saved: alexnet_intel.pth, vgg16_intel.pth, vgg19_intel.pth")

```

Models saved: alexnet_intel.pth, vgg16_intel.pth, vgg19_intel.pth

